

Passa Adiante

Como se decide o que construir
antes de escrever a primeira linha

Este documento conta o que aconteceu entre receber um edital e ter um produto definido. Não é um relatório de features: é o registro de um processo que mudou de ideia três vezes, errou em público, e foi corrigido por perguntas que só um humano faria.

Vem junto o Diário de Bordo completo — nove episódios de trabalho com IA generativa, incluindo os erros — e uma entrevista respondida à mão, sem edição.

AUTOR

Gabriel Vieira Soriano Aderaldo

Ciência da Computação · UNIFOR

PERÍODO

26 a 28 de julho de 2026

18 cerimônias · 9 episódios · 0 linhas de código

O que tem aqui dentro

- Afinal, o que é isso aqui? 3**
 - O que você vai ler aqui 3
- A parte em que o problema é descoberto errado 4**
 - Quatro pessoas, e uma pergunta que ninguém tinha feito 4
 - A geladeira 4
 - E aí veio a provocação que derrubou tudo 5
- O produto que saiu disso 7**
 - Como funciona, em sete passos 7
 - Duas ideias que valem mais que o resto 7
 - O que o produto deliberadamente não faz 8
- Diário de Bordo: nove episódios 9**
 - O que os nove episódios têm em comum 12
- A entrevista, sem edição 13**
 - Sobre trabalhar com IA 13
 - Sobre o que mais irritou 13
 - Sobre as decisões 14
 - Sobre o produto, sem otimismo 14
- O que vem depois 16**

Uma observação sobre como isto foi escrito. Tudo neste documento passou pela mão de uma IA, menos uma coisa: as respostas da entrevista, na última parte. Elas estão exatamente como foram digitadas — erro de digitação, palavrão e frase pela metade inclusive. É a única parte em que a voz não foi filtrada, e é de propósito.

Afinal, o que é isso aqui?

Todo desafio técnico começa igual: você recebe um documento, lê os requisitos, e a vontade é abrir o editor e começar a escrever. É o que quase todo mundo faz.

Este projeto fez o contrário. Passou dois dias inteiros sem escrever uma única linha de código de aplicação — e o que saiu disso são dezoito cerimônias de descoberta de produto, dois documentos de decisão arquitetural, um modelo de domínio, e um produto que mudou de ideia três vezes antes de alguém encostar num teclado.

A pergunta óbvia é: **por quê?** E a resposta está na entrevista no fim deste documento, dita por quem decidiu:

GABRIEL, SOBRE A DECISÃO DE NÃO ESCREVER CÓDIGO

Sinceramente, por que é mais PRÁTICO e mitiga praticamente a maioria dos erros que vão surgir na codificação. [...] se eu NÃO tiver uma ideia muito bem centrada, trabalhada, direta e constante sobre algo... a LLM simplesmente vai fazer 500 coisas que não se conectam em nada.

E a analogia que ele usa para explicar é boa demais para eu resumir:

AINDA ELE

Pensa assim... Começa e escrever uma história, porém você só tem uma frase e vai escrevendo, páginas e páginas sem parar [...] e amanhã le ela: Tu vai perceber que você criou um Onitorrinco, o começo é de uma maneira o meio de outra e o final se quer existir vai ser mais diferente ainda.

O upstream existe para não produzir ornitorrincos.

O que você vai ler aqui

Este documento tem quatro partes, e elas ficam progressivamente mais honestas.

A **primeira** conta como o problema foi descoberto — incluindo a parte em que ele foi descoberto errado, duas vezes. A **segunda** mostra o produto que saiu disso, e o que ele deliberadamente não faz. A **terceira** é o Diário de Bordo: nove episódios de trabalho com IA, cada um deles registrando um erro real e como ele foi pego. A **quarta** é uma entrevista sem edição nenhuma.

Se você só tem tempo para uma, leia a terceira. É onde as coisas dão errado.

A parte em que o problema é descoberto errado

O edital pedia um marketplace de economia circular no campus: estudantes cadastram material que não usam mais, outros encontram. CRUD, vitrine, PWA. Direto ao ponto.

O primeiro instinto foi construir exatamente isso. O segundo foi perguntar para quem seria.

Quatro pessoas, e uma pergunta que ninguém tinha feito

A pesquisa foi pequena e o documento assume isso desde a primeira linha: quatro respondentes, sendo um deles o próprio autor. Não é estatística e nunca foi tratada como tal — o instrumento inteiro foi desenhado para render **comportamento passado**, não intenção declarada.

Por que isso importa. Perguntar “você usaria um app assim?” produz um “sim” que não vale nada. Perguntar “o que você fez com o material do semestre passado?” produz uma história. O questionário aplicado tinha dez perguntas e nenhuma delas era do primeiro tipo.

As respostas convergiram num lugar inesperado. Ninguém falou em falta de lugar para anunciar. O que apareceu foi outra coisa:

P01, EX-ALUNA DE JORNALISMO

já quis doar todas as minhas apostilas do ensino médio porém não consegui ir a fundo para descobrir um local seguro e objetivo para fazer isso

P02, ENGENHARIA DE COMPUTAÇÃO – PERGUNTADO PARA ONDE IRIA DESAPEGAR

Biblioteca?

Aquele ponto de interrogação virou uma persona inteira. Não é rejeição do caminho — é desconhecimento de que existe caminho.

A geladeira

Mas o achado que reorganizou o projeto veio de uma pergunta lateral, feita quase por acaso: **o que exatamente são essas “iniciativas do campus” que você mencionou?**

P04 – E ESTA RESPOSTA MUDOU O PRODUTO

existem uma geladeira velha no ponto de onibus do campus que você pode deixar o que quiser nela, é ótima ideia por que vira uma partilha de conhecimento PÚBLICO, porém, ela é mal cuidada e super apagada de qualquer coisa, se não for alguém que ativamente quis olhar para ela... parece mais lixo na rua. [...] cheguei a deixar minhas apostilas do ensino médio lá uma vez, porém fiquei INSEGURO se realmente foi útil ou eu estava só “espalhando lixo”.

Existe, no campus da UNIFOR, uma geladeira velha num ponto de ônibus onde qualquer pessoa pode deixar material para quem quiser. **Fricção zero.** Você chega e larga.

E mesmo assim não funciona.

O que a geladeira provou

Se o problema fosse “desapegar dá trabalho”, a geladeira teria resolvido. Ela tem a menor fricção fisicamente possível, foi usada, e o problema continuou. **Facilitar o ato de anunciar não é a alavanca.** O que falha é o que acontece depois: o gesto fica sem resposta.

E aí veio a provocação que derrubou tudo

Com o produto desenhado e documentado, veio a pergunta que nenhuma cerimônia do método tinha feito:

GABRIEL, NO MEIO DO PROCESSO

o que vai diferenciar o nosso sistema de um whats app? Sinceramente, seria muito mais barato só fazer a vitrine e deixar os contatos e pronto [...] sendo talvez fazer via instagram seja até melhor... eu tenho a resposta, mas dps digo..

E ele estava certo. Um grupo de WhatsApp **já resolve** a dor da geladeira: você posta a foto do livro, alguém responde, você entrega e sabe que serviu. O canal é identificado e síncrono. A insegurança não existe ali.

A IA respondeu com uma feature. Errou.

A RESPOSTA CERTA, DELE

A vitrine é o produto. [...] o objetivo do projeto é o “DESAPEGA” ou seja é a pessoa passar a diante para outro o que existe e criar uma rede de colaboração entre ALUNOS | Pessoas que frequentam o campos, e no macro? Criar conexões que antes não existiam... [...] uma UNIVERSIDADE é um ecossistema inteiro, e esse projeto deve ajudar nisso, deve aproximar, deve-se estimular.

A diferença é estrutural, não de funcionalidade

Um grupo de WhatsApp **conecta quem já está conectado**. Ele é fechado por definição — você precisa ser adicionado por alguém que já te conhece. O calouro do primeiro semestre não está em nenhum, e é exatamente ele que mais precisa.

Um sistema aberto do campus pode apresentar duas pessoas que não se conhecem. Isso não é uma feature que falta no WhatsApp; é uma propriedade que ele não pode ter.

E isso não jogou o discovery fora — explicou ele. A geladeira falha porque é **anônima**, e anônimo é o oposto de vínculo. **“Fiquei inseguro se foi útil”** não é falta de informação; é falta de pessoa. Se você entrega na mão de alguém que agradece, a insegurança nunca aparece.

O produto que saiu disso

O enquadramento final ficou registrado num documento de decisão arquitetural, e cabe numa frase:

O produto existe para agregar o estudante ao ecossistema da universidade. O item é o pretexto do encontro, não o objeto do serviço.

Como funciona, em sete passos

- 01 Alguém publica um item — título, categoria, se é doação ou venda, e **onde costuma estar no campus**.
- 02 O item aparece na vitrine pública. Qualquer pessoa vê, sem se identificar.
- 03 Quem quer o item toca em **Tenho interesse** — e é isso que revela o contato de quem publicou. O clique não é burocracia: é como se obtém o contato.
- 04 As duas pessoas conversam *fora do produto*, no chat institucional que o campus já tem.
- 05 Quem publicou **reserva** o item para uma delas. O item ganha um selo, e quem vai receber ganha um **código**.
- 06 Encontram-se num ponto conhecido do campus — o DJ, o CC, a biblioteca.
- 07 Quem recebe mostra o código. Quem publicou digita e confirma. **O gesto fecha**.

Os dois passos em cinza acontecem **fora do produto**, e isso é decisão, não lacuna. O campus já tem um chat institucional; construir outro seria pedir que as pessoas adotem um lugar novo para fazer o que já fazem.

Duas ideias que valem mais que o resto

O código, e por que ele existe

Até certo ponto do projeto, “entregue” era uma declaração unilateral: quem publicou marcava, e o sistema acreditava. O problema é que o sistema não sabia de nada — e um estado chamado **Entregue** num sistema que nunca viu entrega nenhuma está mentindo no próprio nome.

O código resolve isso. Ele nasce quando a reserva acontece, vai **só** para quem vai receber, e quem publicou precisa dele para fechar o ciclo. Ninguém fecha sozinho.

O que o código prova – e o que ele não prova

Prova: que as duas pessoas participaram.

Não prova: que o encontro foi presencial. Nada impede mandar o código por mensagem, e o produto não tenta impedir — policiar isso exigiria localização, que o modelo de dados recusa explicitamente.

Ainda assim é uma mudança de patamar: sai de “**uma pessoa afirmou**” para “**as duas participaram**”.

Os pontos de encontro, que resolvem três coisas de uma vez

O anúncio indica onde a pessoa costuma estar — o DJ, o CC, a biblioteca. Lugares conhecidos, movimentados, fáceis de achar.

Segurança

Ninguém marca encontro com um estranho num canto vazio do campus.

Onboarding

O calouro **aprende onde ficam os lugares** enquanto pega um livro emprestado.

Propósito

O encontro deixa de ser abstrato e vira um lugar com nome e história.

O “DJ” é o exemplo perfeito do que significa pertencer ao campus: o apelido vem dos blocos D e J, e por anos foi onde o pessoal de tecnologia se reunia para jogar card game. Veterano sabe o que é. Calouro não. **O produto ensina esse vocabulário** — e essa ideia nasceu de uma entrevista, não de uma reunião: a P01, perguntada sobre o que faltou no questionário, pediu sozinha “**uma pergunta sobre lugares/postos de doação**”.

O que o produto deliberadamente não faz

Chat interno	O campus já tem um. Construir outro é pedir que adotem um lugar novo para o que já fazem.
Avaliação de pessoas	Não responde à objeção real, que é sobre o material ter bom cuidado. Saber quem é a pessoa não diz como ela vai tratar a coisa.
Notificações	O uso é episódico — duas ou três vezes por ano. Avisar quem abre o app tão pouco é incômodo, não serviço.
Busca por texto	Não foi pedida pelo edital e não apareceu em nenhuma entrevista.
Verificar quem é da UNIFOR	Não há como. E o produto assume isso: nenhuma tela afirma identidade, porque nenhuma pode.

Diário de Bordo: nove episódios

Esta é a parte em que as coisas dão errado.

Cada episódio abaixo registra um momento em que a IA errou — ou em que o processo errou — e como o erro foi encontrado. Nenhum foi escrito depois, para parecer bem: todos foram registrados no dia, com o prompt original preservado sem maquiagem.

Por que isso é o mais importante do documento. Um diário em que a IA nunca erra não convence ninguém que já usou uma. O que separa uso maduro de uso ingênuo não é a ausência de erro — é ter método para encontrá-lo antes que ele vire código.

EP-001

A escolha do task runner

Quando não há dado, admita que não há

A primeira decisão do projeto foi qual ferramenta usar para automatizar tarefas. A IA produziu uma comparação convincente entre alternativas — e nenhuma daquelas afirmações tinha sido verificada.

O que ficou: decisão sem dado é chute com formatação bonita. O projeto passou a exigir que toda afirmação viesse rotulada como **verificado**, **documentado** ou **inferido** — e a regra sobreviveu até o último episódio.

EP-002

A correção que estava errada

Conhecimento de modelo tem prazo de validade

A IA tentou “corrigir” a versão do Docker Compose declarada no projeto, com confiança total, usando informação desatualizada. A correção teria quebrado o que funcionava.

O que ficou: versões entram no projeto como número explícito, nunca como “a mais recente”. O modelo não sabe em que ano está.

EP-003

Documentei a ferramenta errada

Configurar sem verificar o alvo

Foram escritas configurações detalhadas de permissão para uma ferramenta que este projeto não usa e nunca usaria. Documentação impecável, produto errado.

O que ficou: antes de configurar, confirmar qual ferramenta está em jogo. Parece óbvio; foi preciso errar para virar regra.

EP-004

O portão que eu declarei testado três vezes

E que não fechava

Um mecanismo de proteção foi declarado testado — três vezes, com oito casos passando — e tinha um contorno alcançável com duas edições comuns. Os testes viviam em mensagens de chat e morriam com elas; nada os re-executava.

O que ficou: uma suíte de testes de verdade, com 54 casos, ligada ao portão de verificação. E o hábito de **validar por sabotagem**: quebrar o mecanismo de propósito para ver se o teste pega.

EP-005

Quando a teoria confirma a prática

O episódio sem erro

Nem todo episódio é sobre falha. Aqui, a fundamentação teórica encontrada depois confirmou uma decisão que já tinha sido tomada por intuição — e o registro serve para isso: mostrar que intuição informada às vezes chega antes da bibliografia.

EP-006

Três stacks num dia

E a pergunta que estava errada

Quatro agentes especializados debateram uma escolha de tecnologia por horas, com testes reais, medições e benchmarks. Um deles abriu o edital, leu a seção de critérios de avaliação, e mostrou que **nenhum dos eixos pontuava escolha de stack**.

E as medições tinham um problema pior: o benchmark que “validou” uma das opções reportou **2062 requisições por segundo, zero erros** — medindo contra banco vazio, com o método de escrita retornando erro 500 em todos os casos. O caminho que quebrava não estava no teste.

O que ficou: investigação profunda não protege contra a pergunta errada. E existe uma diferença enorme entre **medir** e **concluir**.

EP-007

A citação pela metade

E a geladeira que ninguém tinha perguntado

Este é o erro mais desconfortável do projeto, e não é uma alucinação.

Uma resposta de entrevista dizia: “**Já sim, e já passei mais de uma vez, porém tem algumas pequenas iniciativas que existem no campus que eu decidi NÃO usar...**”. A IA publicou **só a segunda metade** e concluiu que aquilo refutava uma hipótese — quando a primeira metade dizia exatamente o contrário.

Não foi invenção: foi **uma citação real, cortada no ponto em que apoiava a conclusão que estava sendo construída**, dentro de um documento que se apresentava como evidência.

Por que esse tipo de erro é pior que alucinação

API inventada quebra no primeiro teste. Citação truncada é publicada, vira base de decisão, e **não dá para pegar lendo** — porque o documento fica bom. Só dá comparando com a fonte.

Como foi pego: um papel cético foi instruído a conferir cada citação contra o arquivo bruto. Rodou uma busca pela frase e recebeu zero resultados.

O que ficou: texto entre aspas tem que ser localizável por busca na fonte. Ortografia se corrige com colchetes, nunca dentro das aspas — e a IA tinha “corrigido” acentos dentro de citações, o que quebrou a busca e fez **dois analistas independentes** concluírem que as citações não tinham origem.

EP-008

Dez cerimônias sem perguntar por quê

O erro que não foi de execução

Dez cerimônias de método, doze documentos, um comitê de sete especialistas auditando tudo com fonte citada. E ninguém, em momento nenhum, perguntou **por que este produto deveria existir**.

A pergunta veio do humano — a provocação do WhatsApp — e derrubou mais em três linhas do que horas de auditoria.

O que ficou: a pergunta de existência não está em nenhuma cerimônia do método. O livro tem declaração de suposições, hipóteses, MVP, experimentos — e todos pressupõem que o produto deve existir. **Rigor metodológico pode disfarçar produto sem razão de ser, e disfarçou.**

E uma lição sobre o próprio diário: a citação mais importante do episódio não estava no registro, porque tinha sido escrita por um caminho que o mecanismo de captura não observava. Um instrumento que perde justamente a intervenção decisiva é um instrumento com ponto cego — e só se descobre indo procurar a frase para citá-la.

EP-009

O upstream fechado

E o que os especialistas pegaram

O trecho final produziu mais correções ao trabalho da própria IA do que qualquer outro — e todas vieram de alguém **abrir o arquivo e conferir**, nunca de alguém pensar mais.

A pior: ao reescrever os critérios de aceite, a IA criou uma contradição com o modelo de domínio e não conferiu. Três documentos passaram a discordar. Um especialista encontrou, e o argumento que ele usou para escolher entre as versões **era melhor que o da IA** — a decisão certa foi tomada por acidente.

O que ficou: o modo de falha não é falta de inteligência. É confiar na memória do que se escreveu.

E uma nota boa: o mecanismo de captura corrigido no episódio anterior registrou as duas citações mais importantes deste. **A lacuna foi fechada a tempo de servir ao episódio seguinte.**

O que os nove episódios têm em comum

9

episódios

7

com erro real

0

encontrados por reflexão

Nenhum erro deste projeto foi encontrado por alguém pensando com mais cuidado. Todos foram encontrados por alguém **comparando o que estava escrito com a fonte** — rodando o comando, abrindo o arquivo, conferindo a citação.

É por isso que o projeto acumulou tanta infraestrutura de verificação: uma suíte de testes para os próprios mecanismos de automação, um registro automático de tudo que foi pedido à IA, e a regra de que toda afirmação vem rotulada pelo grau de confiança.

A entrevista, sem edição

Esta última parte é a única do documento em que a voz não passou por nenhum filtro. As perguntas foram feitas pela IA; as respostas foram escritas à mão e estão exatamente como foram digitadas.

Nada aqui foi corrigido — nem digitação, nem concordância, nem as críticas diretas à ferramenta que escreveu o resto deste documento. Editar teria sido perder o que ela tem de melhor.

Sobre trabalhar com IA

SOBRE O QUE A IA É, NO PROCESSO DELE

Hoje em dia encaro a IA como meu: “COM O PERDÃO GIGANTE” da expressão, meu estagiário pessoal, onde eu dito tudo que quero e até discuto várias vezes com ele para chegar em algum lugar. Porém também ela é muito importante no meu processo [...] que ela funciona como um pato de borracha a cada minuto e segundo, pois ela é tão boa em NÃO entender porra nenhuma do que eu quero dizer ou escrevo que vira um exercício de validação de se eu estou REALMENTE seguro na minha ideia inicial.

E, SEM MEIAS PALAVRAS

A IA (desculpa claude se você ler isso), mas você não PENSA, você não tem raciocínio e muito menos ENTENDE um texto. É só um eterno cuspidor de STRINGs probabilísticas bem feito.

PERGUNTADO SE CONFIA MAIS OU MENOS NA IA HOJE

Eu? nunca “confiei”, você é uma ferramenta de produtividade... se você errou, a culpa é minha. Essa pergunta é a mesma coisa que dizer: “Quando você confiou no gerador de código do build runner do flutter?”

Sobre o que mais irritou

PERGUNTADO SOBRE O MOMENTO DE MAIOR IRRITAÇÃO

Com toda a certeza quando eu não aguentava mais falar que não queria saber sobre .NET e quando percebi que você não escreveu NADA com nada do discovery 5 até o 10 e eu tive refazer tudo.

SOBRE POR QUE PRECISOU CORTAR OS ASSUNTOS DE PRAZO E TECNOLOGIA

Sinceramente, você estava fazendo o que mais faz, pega valores determinísticos para seguir no caminho de maior probabilidade. [...] Sendo assim eu cortei só pra tu não gastar token que eu pago me perguntando pela 1000 vez, quando era pra entregar isso, sendo que nem nossão de tempo tu tem.

Sobre as decisões

SOBRE POR QUE SEGUROU A RESPOSTA DA PROVOCAÇÃO

Queria saber o que já tínhamos como contexto e o que tínhamos escrito nos documentos já feitos e onde isso ia te levar de sugestão... basicamente se tu fosse até a lua e dps pulasse para plutão queria dizer que tava tudo muito mal feito.

SOBRE A GELADEIRA – E DISCORDANDO DA PRÓPRIA IA QUE ESCREVEU ESTE DOCUMENTO

Eu não acho que ela seja a resposta ainda agora, mas tu tem tendencia de pegar umas coisas soltas, juntar umas frases de efeito como se fosse “A VERDADE”, então eu só ignoro, não vale a pena “corrigir” uma máquina inanimada como se fosse um ser humano.

SOBRE DE ONDE VEIO A ANALOGIA DO MAHJONG

costumo usar muita analogia para descrever as coisas pois a IA tem uma tendencia de entender melhor processos não abstratos documentados do que eu ter que explicar processos meio inacabados que tenho na cabeça [...] e... não eu não jogo Majong, porém assisti um anime chamado “SAKI” que é sobre majong e lembrei do tipo de jogada.

Sobre o produto, sem otimismo

PERGUNTADO SE ACREDITA QUE ALGUÉM USARIA

Sinceramente ? Não sei, mas eu acho que usaria... sempre faço pensando. Eu usaria? se a resposta é “não”, costumo re-avaliar se vale a pena fazer

PERGUNTADO DO QUE TEM MAIS MEDO

Talvez a questão do produto em si, tenho medo virar uma grande “vitrine” e o resto a galera vai fazer tudo via Zap

PERGUNTADO SE INCOMODA A PESQUISA NÃO TER NENHUM CALOURO – QUE É QUEM O PRODUTO DIZ ATENDER

MUITO

PERGUNTADO SE HÁ ERROS DA IA AINDA NÃO ENCONTRADOS

Vários, só que eu ainda não sei onde estão... quando eu continuar o processo vou achar, quando o usuário usar o sistema ele tbm vai achar... faz parte.

E A ÚLTIMA LINHA DA ENTREVISTA, SOBRE O QUE AINDA INCOMODA

Sinceramente não estou satisfeito ainda com NADA da stack... acho que vou rever ela jaja.

O que vem depois

O upstream fechou. O que existe agora é um produto definido do problema até a frase que aparece quando alguém digita o código errado — e nenhuma linha de código de aplicação.

O que este documento não é

Não é uma demonstração de que o processo funcionou. Isso só se sabe depois de construído, e o próprio autor diz na entrevista que não sabe se alguém usaria.

É o registro de **como se decidiu**, com os erros no meio. Se o produto der errado, este documento continua sendo verdadeiro — e provavelmente vai mostrar onde a decisão errada foi tomada.

A próxima fase é a esteira de produção: transformar dezoito cerimônias em software que sobe, com testes escritos antes do código. O material está pronto — cada critério de aceite foi escrito como comportamento observável, exatamente para virar teste.

E se algo der errado lá, vai virar o episódio dez.

Repositório completo, com todos os documentos citados: github.com/GabrielAderaldo/desafio-vortex

Jornadas e wireflows: figma.com/community/file/1663876864232938978

Documento gerado em Typst. Os nove episódios completos estão em `docs/ai-log/`, e a entrevista integral em `docs/ai-log/entrevista-01-upstream.md`.